



1. Resumen del proyecto

El proyecto consiste en el diseño y desarrollo de una placa electrónica para un robot capaz inspeccionar y operar un Cubo de Rubik 3x3 de manera automatizada. El sistema utiliza una cámara para capturar el estado del cubo y un microcontrolador ESP32-S3-WROOM-1 para procesar la información y controlar servomotores encargados de realizar los movimientos necesarios. Además, se incorpora una pantalla LCD manejada por expansores I2C para la visualización del estado del sistema con el tiempo aproximado de demora, y conectividad WiFi para la comunicación con una computadora externa.

2. Abstract

The project consists of designing and developing an electronic board for a robot capable of automatically inspecting, and operating a 3x3 Rubik's Cube. The system uses a camera to capture cube's state and an ESP32-S3-WROOM-1 microcontroller to process the information and control servomotors responsible for performing the required movements. Additionally, an LCD screen controlled by I2C expanders is included to display system's status with an approximate delay time, and WiFi connectivity is provided for communication with an external computer.

3. Estado del arte

Diversos trabajos han abordado la resolución automática del cubo Rubik mediante la integración de visión artificial, algoritmos de resolución y sistemas robóticos.

[1] David Gilday (ingeniero de ARM Holdings) es el creador de MindCub3r que es un robot de dos actuadores ("brazo/jaulaz "plataforma giratoria") utilizando LEGOs para demostrar la eficiencia de los procesadores embebidos ARM en ferias tecnológicas internacionales, y en este blog demuestra como logró manipular cubos complejos sin usar decenas de motores.

D. Gilday, "How I Created the ARM Powered Android LEGO Cube Solving Robot," *ARM Developer Community Blogs*, Sep. 11, 2013. [Online] Disponible en:

<https://developer.arm.com/community/arm-community-blogs/b/internet-of-things-blog/posts/how-i-created-the-arm-powered-android-lego-7x7x7-cube-solving-robot>.

[2] Uno de los algoritmos más utilizados para la resolución del cubo Rubik es el método de dos fases desarrollado por Herbert Kociemba. Este algoritmo permite obtener soluciones eficientes en un número reducido de movimientos (aproximadamente 20 movimientos), siendo ampliamente utilizado en aplicaciones robóticas debido a su rapidez y confiabilidad.

H. Kociemba, "Two-Phase Algorithm for Solving Rubik's Cube," 2010. [Online] Disponible en:

<https://kociemba.org/cube.htm>

[3] En el ámbito de la robótica, existen implementaciones que combinan visión artificial y control de actuadores. Por ejemplo, en el trabajo presentado por Owolabi et al., se desarrolla un robot capaz de resolver el cubo Rubik utilizando procesamiento de imágenes para identificar colores y motores para ejecutar los movimientos.

O. Owolabi, A. O. Adewumi, and O. O. Oladipo, "Development of a Rubik's Cube Solving Robot," *International Journal of Engineering Research & Technology (IJERT)*, vol. 8, no. 11, 2019.

[4] Asimismo, en proyectos académicos se ha explorado el uso de plataformas embebidas como Raspberry Pi para integrar la captura de imágenes, el procesamiento y el control de actuadores en un solo sistema, destacando la viabilidad de soluciones compactas y de bajo costo.

S. Tharayil and A. K. S, "Design and Implementation of Rubik's Cube Solving Robot using Raspberry Pi," *International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering*, vol. 6, no. 6, 2017.

Estos trabajos demuestran que la resolución automática del cubo Rubik es un problema ampliamente estudiado, donde la combinación de algoritmos eficientes, visión artificial, control de actuadores y una buena elección de la mecánica permite desarrollar sistemas funcionales y de bajo costo. El presente proyecto se basa en estos enfoques, incorporando el diseño de una PCB dedicada para integrar todos los módulos en un sistema compacto.

4. Descripción

El proyecto "*Scrambler Bot*" propone el desarrollo de un prototipo comercial mecatrónico orientado a resolver los cuellos de botella logísticos y los errores humanos (*mis-scrambles*) presentes en el desordenado manual de los Cubos de Rubik durante competiciones oficiales de la WCA (*World Cube Association*) y en las sesiones de enseñanza en academias de *speedcubing*. A diferencia de las costosas alternativas industriales de múltiples motores, este sistema implementa una arquitectura cinemática optimizada de una base giratoria y un brazo manipulador controlada por servomotores de alto torque. Esta solución eficiente se gestiona a través de una placa de circuito impreso (PCB) de diseño propio gobernada por un microcontrolador ESP32-S3, el cual no solo ejecuta las rutinas mecánicas con precisión, sino que integra un módulo de cámara (OV2640) para validar ópticamente el estado del cubo mediante algoritmos de visión artificial, garantizando que el desorden final coincida matemáticamente con el estándar oficial exigido.

La motivación principal detrás de esta iniciativa radica en el gran desafío multidisciplinario que representa, exigiendo la integración armónica de diseño electrónico, programación de sistemas embebidos y diseño. De esta forma, el proyecto busca transformar un problema logístico documentado dentro de la comunidad de speedcubers en un producto de hardware funcional, elegante y asequible, demostrando cómo una ingeniería eficiente puede generar soluciones con un claro potencial de escalabilidad comercial.



5. Objetivos

a. General

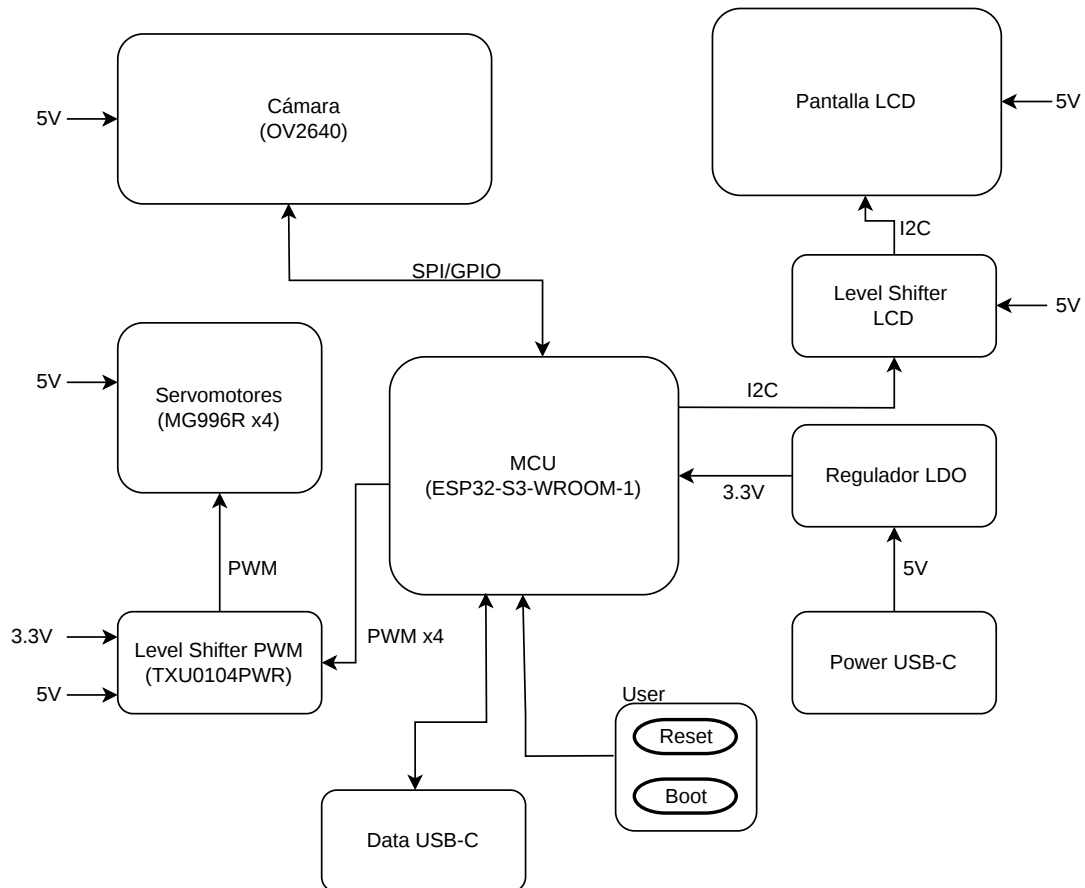
Diseñar la placa de circuito impreso para el control de un sistema automatizado de resolución del Cubo de Rubik.

b. Específicos

1. Evaluar las alternativas mecánicas para la manipulación del cubo.
2. Diseñar la arquitectura electrónica general del sistema para la alternativa elegida.
3. Seleccionar los componentes adecuados para la implementación de la placa PCB.
4. Realizar el diagrama de bloques del sistema con sus protocolos de comunicación y alimentación correspondientes.
5. Elaborar el esquemático y conexiones de los componentes electrónicos de la placa.
6. Diseñar la PCB integrando el microcontrolador, la cámara, la pantalla LCD, la alimentación y las salidas para actuadores.
7. Implementar las rutinas de software para el control PWM de los actuadores y el procesamiento de la interfaz de visión, validando la comunicación inalámbrica mediante WiFi entre el sistema electrónico y una computadora externa.

6. Esquema general del proyecto

Diagrama en bloques



Descripción de las partes

1. Microcontrolador: ESP32-S3-WROOM-1-N8R8

Encargado de coordinar el funcionamiento general del sistema, procesar la información recibida de los periféricos y generar las señales de control para los actuadores.

Características relevantes:

- SoC ESP32-S3 con conectividad WiFi integrada.
- Alimentación de 3.3 V.
- Múltiples GPIO disponibles.
- Soporte para interfaces SPI, PWM, I2C, UART y USB.
- Capacidad suficiente para el manejo de periféricos y control del sistema.

2. Módulo cámara OV2640

Se utiliza para la captura de imágenes del cubo Rubik. Su información permite identificar la disposición de colores en las caras del cubo y generar la secuencia de resolución.

Características relevantes:

- Alimentación de 5V.
- Protocolo SPI.

3. Pantalla LCD

Proporciona una interfaz visual para el usuario, permitiendo mostrar estados del sistema, y viene acompañado de un *backpack PCF8574* para que pueda ser controlado mediante el protocolo I2C.

Características relevantes:

- Alimentación de 5V.
- Preset para ajustar el contraste.
- Protocolo I2C.

4. Level Shifter para LCD

Permite adaptar los niveles lógicos entre el ESP32, que opera a 3.3 V, y la pantalla LCD, que opera a 5 V, garantizando compatibilidad en la comunicación I2C.

5. Level Shifter PWM

Convierte las señales PWM del microcontrolador a niveles apropiados para el control de servomotores alimentados a 5 V.

6. Servomotores MG996R

Son los actuadores encargados de manipular físicamente el cubo Rubik. La placa fue diseñada para soportar hasta cuatro salidas PWM, aunque en la implementación actual se utilizan dos servomotores.

Características relevantes:

- Alimentación de 5V.
- Corriente de bloqueo de 2,5A.

7. Módulo USB-C (Alimentación)

Proporciona la entrada principal de energía del sistema. Incluye protección, filtrado y regulación de voltaje para alimentar tanto la lógica a 3.3 V como los periféricos que operan a 5 V.

8. Módulo USB-C (Datos)

Interfaz de comunicación bidireccional encargada de la carga del firmware en el microcontrolador ESP32-S3 y la depuración del código mediante comunicación serie en tiempo real.



6.1. Product Breakdown Structure

1. Scrambler Bot

1.1. Hardware

1.1.1. PCB

1.1.1.1. MCU

1.1.1.2. Level Shifter

1.1.1.2.1. PWM

1.1.1.2.2. LCD

1.1.1.3. I2C Backpack

1.1.1.4. Fuente de Alimentación

1.1.1.4.1. Alimentación

1.1.1.4.1.1. Power USB-C

1.1.1.4.2. Datos

1.1.1.4.2.1. Data USB-C

1.1.1.4.2.2. Protección EDS

1.1.1.4.3. Protección de Retroalimentación de Corriente

1.1.1.5. Regulador LDO

1.1.2. Periféricos Externos

1.1.2.1. Módulo Cámara OV2640

1.1.2.2. Pantalla LCD

1.1.2.3. Servomotor MG996R

1.2. Software

1.2.1. Algoritmo de resolución

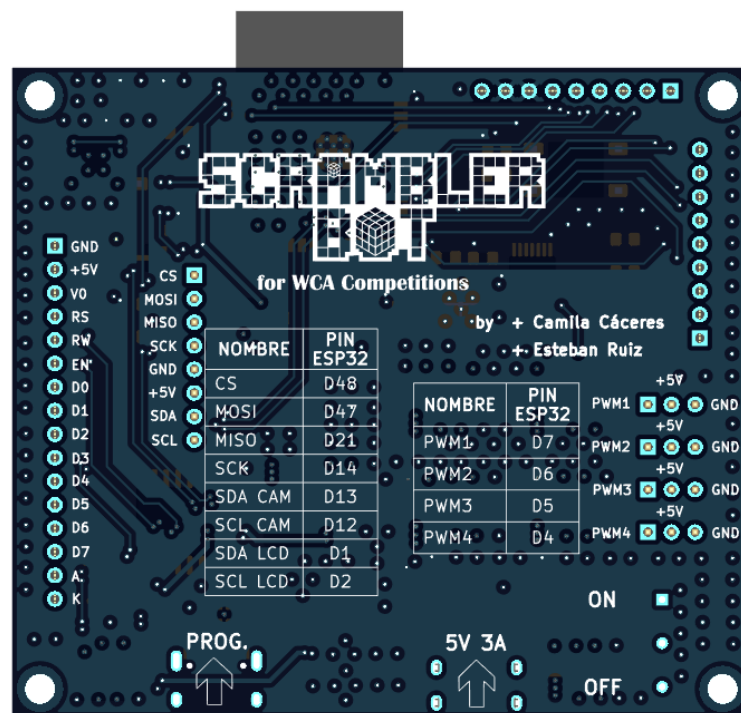
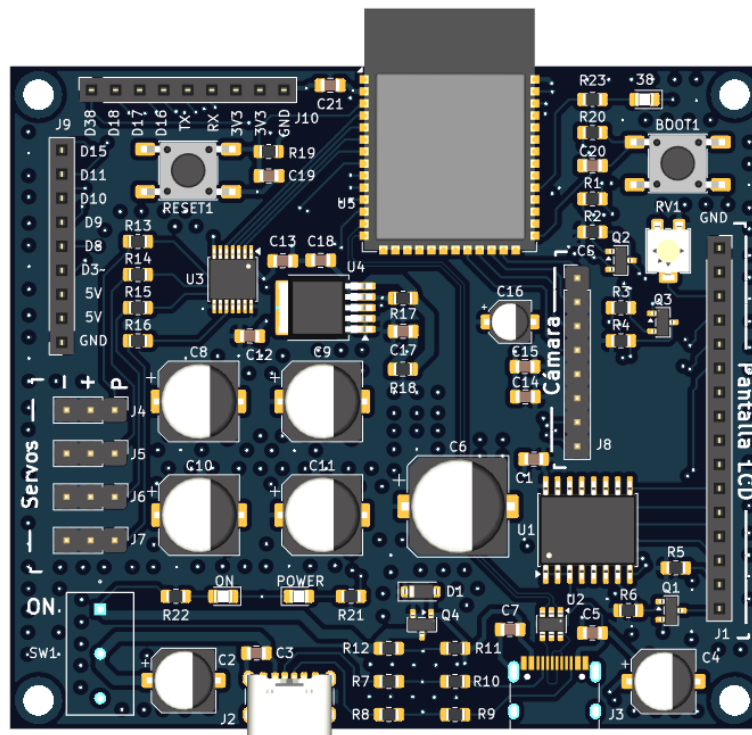
1.2.2. Código para controlar periféricos externos.

1.2.3. Interfaz de usuario

7. Diseños

a. Electrónico

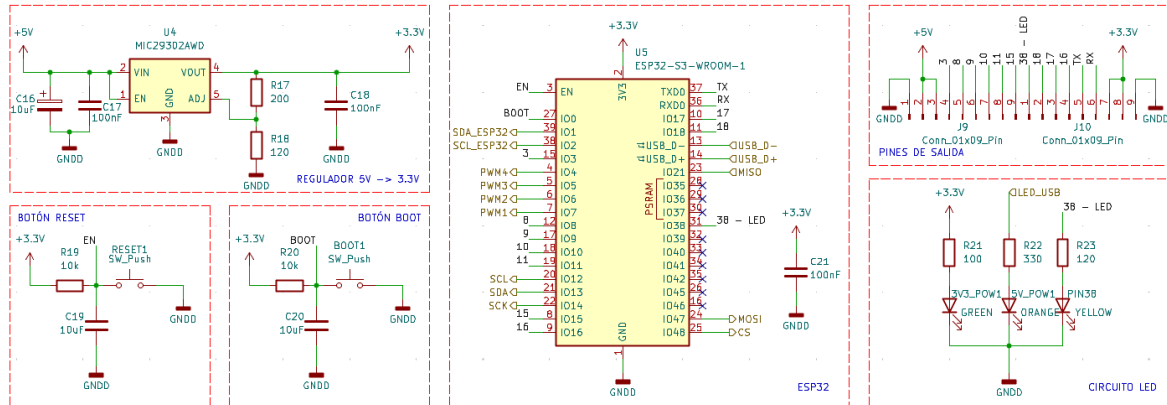
Diseño PCB



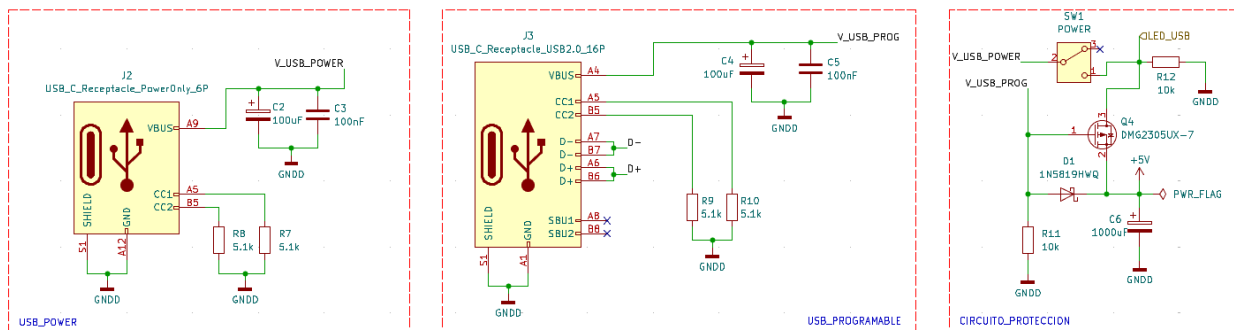
Esquemáticos

A continuación se presentan los distintos bloques esquemáticos que componen el sistema, junto con una breve descripción de la función de cada uno.

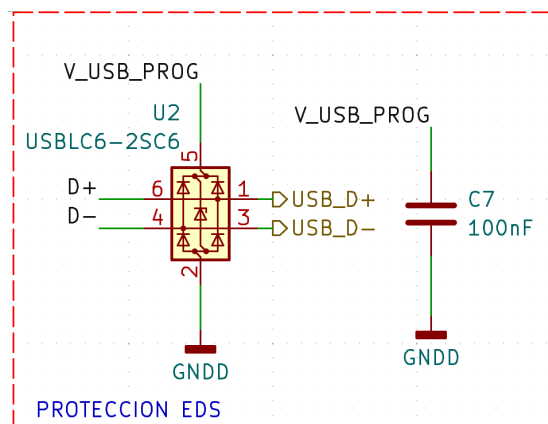
1. MCU, control principal y Regulador LDO



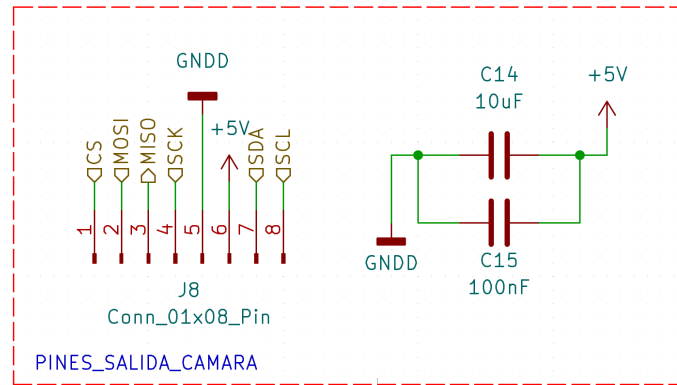
2. USB-C de Datos y Alimentación



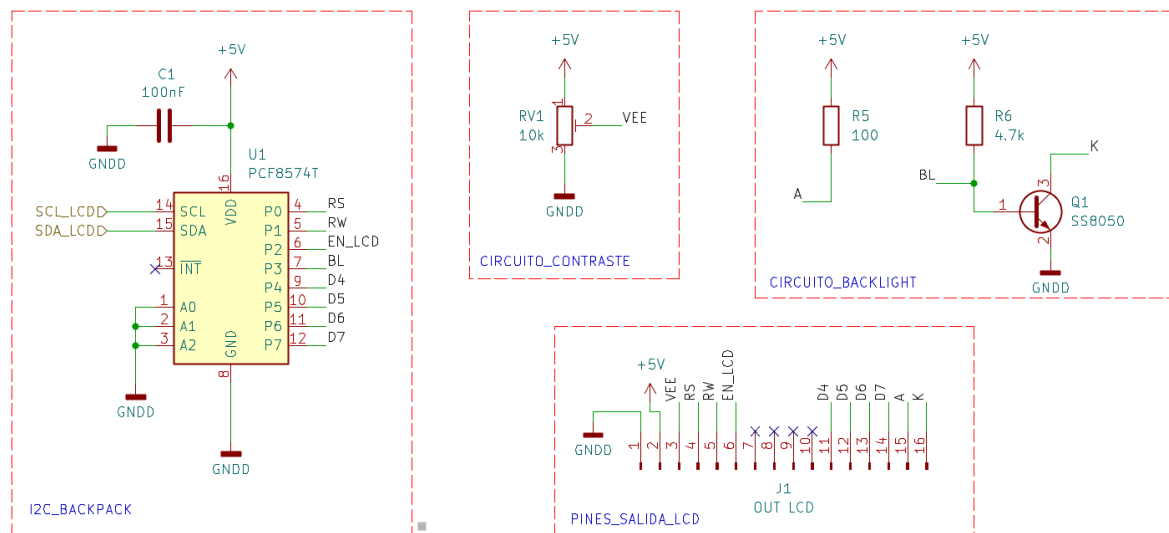
3. Protección ESD en USB



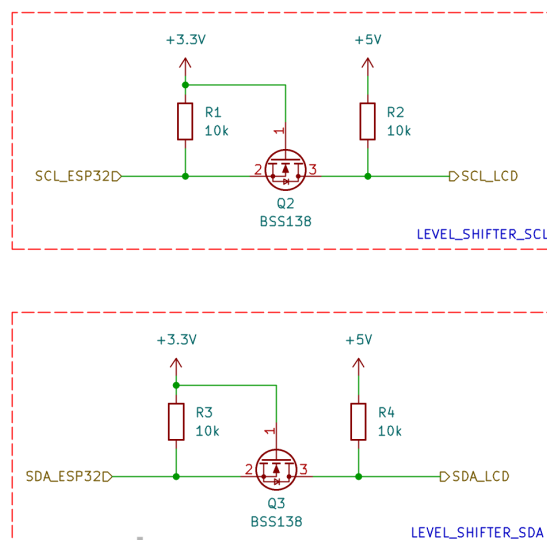
4. Cámara OV2640



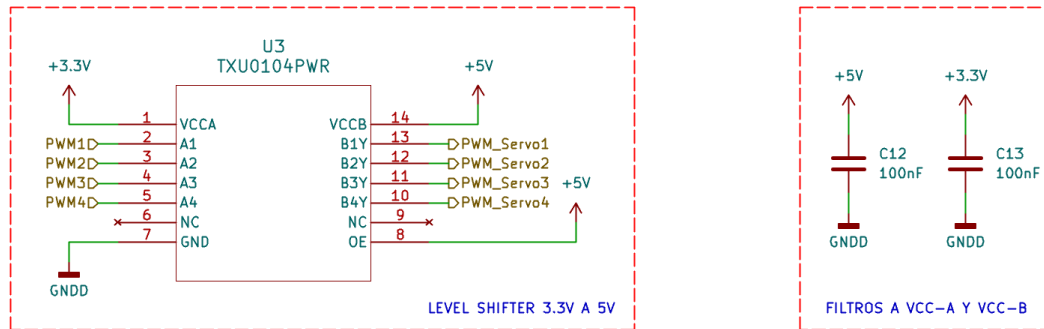
5. Pantalla LCD



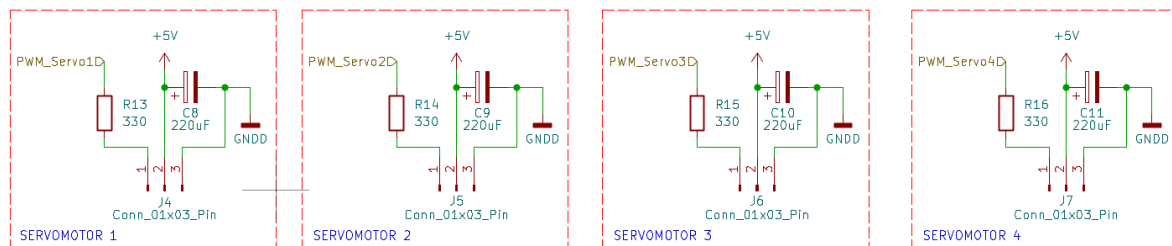
6. Level Shifter LCD



7. Level Shifter PWM



8. Salidas para servomotores



Descripción:

1. USB-C Datos y Alimentación

- Conector USB-C: Entrada principal de energía del sistema (5V, 3A).
- Líneas D+ y D-: Utilizadas para programación y comunicación del ESP32.
- Capacitores de filtrado: Estabilizan la línea de alimentación de 5V.

2. Protección de alimentación

- Diodo Schottky: Evita la retroalimentación de corriente.
- MOSFET de protección: Protege frente a sobrecorrientes.
- Elementos de filtrado: Reducen ruido eléctrico en la alimentación.

3. Protección ESD en USB

- Protección para líneas D+ y D- mediante diodos ESD.
- Reduce el riesgo de daño por descargas electrostáticas.

4. Regulador LDO

- MIC29302AWD-TR: Regulador encargado de convertir 5V a 3.3V.
- Capacitores de entrada y salida: Mejoran la estabilidad y reducen ruido.
- Red resistiva: Ajuste de tensión de salida del regulador.



5. Microcontrolador ESP32-S3

- ESP32-S3-WROOM-1: Unidad central de procesamiento del sistema.
- Interfaces de comunicación: SPI, PWM, I2C, UART y GPIO.
- Capacitores de desacoplo: Estabilidad en la alimentación del microcontrolador.

6. Botones de control

- Botón RESET: Reinicio del microcontrolador.
- Botón BOOT: Permite ingresar en modo de programación.

7. Sistema de cámara

- Módulo OV2640: Captura imágenes del cubo Rubik.
- Interfaz SPI/GPIO: Comunicación con el ESP32.
- Pines dedicados: CS, MISO, MOSI, SCK, SDA y SCL.

8. Pantalla LCD

- Interfaz I2C mediante backpack.
- Utilizada para visualización del estado del sistema.
- Conexión a través de level shifter.

9. Level Shifter LCD

- Conversión de niveles lógicos de 3.3V a 5V.
- Permite comunicación segura entre ESP32 y LCD.
- Implementado con transistores MOSFET.

10. Level Shifter PWM

- TXU0104PWR: Conversor de nivel lógico para señales PWM.
- Convierte señales de 3.3V a 5V para control de servos.
- Soporta hasta 4 canales PWM.

11. Servomotores

- Salidas PWM para hasta 4 servomotores.
- El sistema utiliza actualmente 2 servos (MG996R).
- La PCB está preparada para expansión futura.

12. Pines de conexión

- Pines rotulados con +5V, +3.3V, GND y señales de control.



8. Costos

a. Materiales y herramientas compradas

Descripción	Cantidad	Precio (USD)	Total (USD)
ESP32-S3-WROOM-1-N8R8	1	6.23	6.23
Regulador LDO de 5V a 3.3V MIC35302WD-TR	1	1.96	1.96
LED Naranja (USB _P OWER)HSML – C170 – T0000	1	0.91	0.91
LED Verde (GENERAL _P OWER)HSMG – C170	1	0.28	0.28
LED Amarillo (Pin 38) HSMY-C170	1	0.3	0.3
Puente de diodos USBLC6-2SC6	1	0.36	0.36
Switch BOOT y RESET 430481025816	2	0.57	1.14
Resistencia 330 CRCW0805330RFKEA	5	0.1	0.5
Resistencia 100 CRCW0805100RKNEAIF	2	0.61	1.22
Resistencia 200 CRCW0805200RKNEAIF	1	0.61	0.61
Resistencia 120 CRCW0805120RFKEA	2	0.1	0.2
Resistencia 10k CHP0805AFX-1002ELF	8	0.38	3.04
Capacitor 100nF GCM21BR72A104KA37K	10	0.18	1.8
Capacitor 10uF GCM21BC71C106KE36K	3	0.41	1.23
Capacitor 10uF EEE-FK1C100AR	1	0.45	0.45
Regla de 9 pines hembra M20-7820946	2	1.3	2.6
USB-C Alimentacion USB4125-GF-A	1	0.7	0.7
Capacitor 100uF EEE-FP1C101AP	2	0.63	1.26
Resistencia 5.1k CRCW08055K10FKEA	4	0.1	0.4
Capacitor 1000uF EEE-FN1C102UP	1	1.12	1.12
Switch 7101SYZQE	1	7.52	7.52
USB-C Datos USB4105-GF-A	1	0.94	0.94
Diodo Schottky 1N5819HWQ-7-F	1	0.43	0.43
MOSFET DMG2305UXQ-7	1	0.36	0.36
Regla de 16 pines hembra 61301611821	1	0.69	0.69
Regla de 40 pines macho 61304011121	1	1.53	1.53
MOSFET BSS138LT3G	2	0.28	0.56
PCF8574DWR	1	1.99	1.99
Capacitor 220uF EEE-FK1C221P	4	0.7	2.8
Level Shifter de 4 canales TXU0104PWR	1	0.99	0.99
Trimmer 10k 3314G-1-103E	1	1.58	1.58
Transistor SS8050-HF	1	0.29	0.29
Resistencia 4,7k CRCW08054K70FKEA	1	0.10	0.10
Modulo Arducam OV2640	1	25.99	25.99
Costo total			72.08 USD

b. Materiales y herramientas existentes

Componente	Cantidad
Servomotor MG996R	4
Pantalla LCD 20x4	1
Flux en pasta	1
Estaño en pasta SN67PB37	1
Malla de cobre	1

9. Montaje, horneado y pruebas**9.1. Procedimiento de montaje y horneado de la PCB**

El montaje de la placa se realizó en la facultad utilizando el método de soldadura por pasta de estaño y horno de refusión. La PCB y los componentes fueron adquiridos por separado, incluyendo también el stencil correspondiente al diseño de la placa.

Debido a que el fabricante JLCPCB envía un mínimo de cinco placas iguales por pedido, se utilizaron algunas de las placas sobrantes para facilitar el proceso de ensamblado. Tres placas fueron fijadas sobre la mesa de trabajo con cinta adhesiva, formando una base de apoyo que permitió inmovilizar la PCB principal y mantenerla alineada durante la aplicación de la pasta de estaño.

El procedimiento seguido fue el siguiente:

1. Se colocó la PCB principal en el centro de la base de apoyo y se alineó cuidadosamente el stencil con las huellas de los componentes SMD.
2. Una vez alineado el stencil, se aplicó estaño en pasta sobre la superficie.
3. Con ayuda de una tarjeta, se distribuyó la pasta de estaño sobre el stencil. Primero se realizó el arrastre con un ángulo aproximado de 45 grados, luego con un ángulo cercano a 30 grados para mejorar la distribución y finalmente se pasó la tarjeta a 90 grados para retirar el exceso de pasta.
4. Luego de aplicar la pasta, se retiró cuidadosamente el stencil para evitar desplazar el estaño depositado sobre las huellas.
5. Posteriormente, se colocaron los componentes SMD sobre la placa utilizando pinzas y verificando su orientación, especialmente en componentes polarizados como diodos, LEDs, capacitores electrolíticos, conectores USB-C y circuitos integrados.
6. Una vez colocados los componentes, la PCB fue llevada al horno de refusión disponible en la facultad.
7. En el horno se seleccionó la curva de temperatura correspondiente al tipo de estaño en pasta utilizado. El proceso incluyó una etapa inicial de precalentamiento y posteriormente una curva de temperatura que alcanzó aproximadamente los 240 °C durante

un tiempo cercano a 8 minutos, permitiendo la correcta fusión de la pasta de estaño y el soldado de los componentes SMD.

8. Finalizado el horneado, se dejó enfriar la placa y se realizó una inspección visual de las soldaduras, verificando posibles cortocircuitos, componentes desplazados o uniones incompletas.

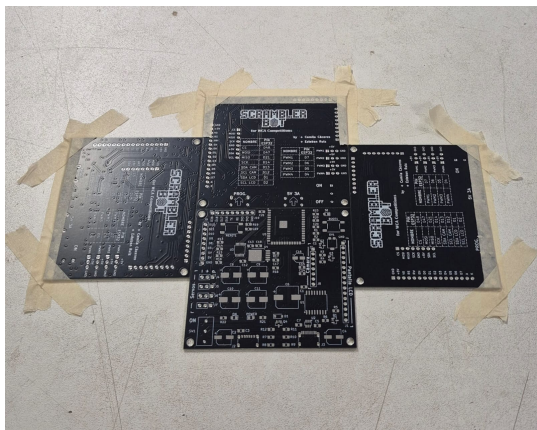


Figura 1: Inmovilización de la PCB utilizando placas sobrantes del pedido.

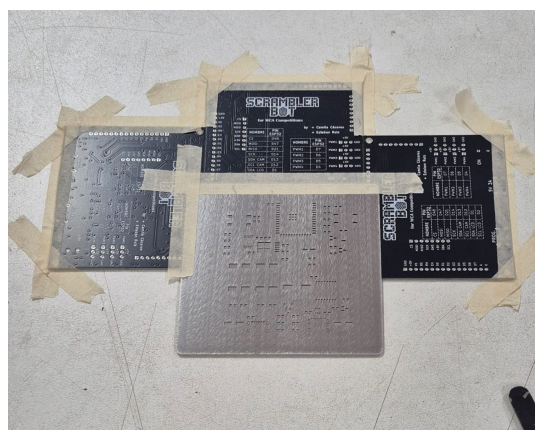


Figura 2: Aplicación de pasta de estaño utilizando stencil.

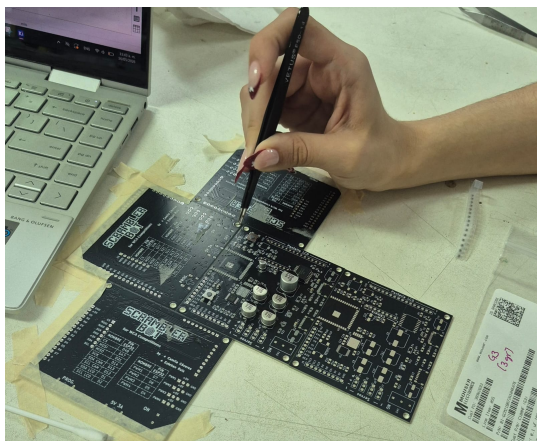


Figura 3: Colocación manual de componentes SMD utilizando pinzas.



Figura 4: Proceso de horneado de la PCB en horno de refusión.



Figura 5: Curva de temperatura utilizada durante el proceso de soldadura SMD.

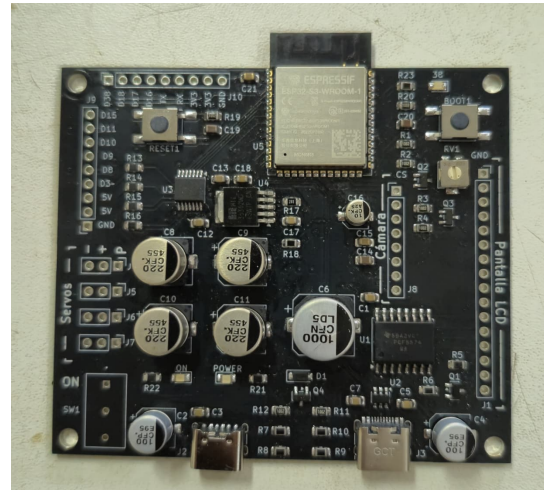


Figura 6: PCB luego del proceso de montaje y horneado.

9.2. Problemas encontrados y soluciones aplicadas

Durante el montaje y las primeras verificaciones de la placa se identificaron algunos problemas que fueron corregidos manualmente.

1. Cortocircuito en conectores USB-C

Se detectaron cortocircuitos entre algunas patas de ambos conectores USB-C. Este problema se debió al exceso de estaño acumulado entre pines durante el proceso de soldadura.

Para solucionarlo, se inspeccionó la zona utilizando un microscopio, se aplicó flux sobre los pines afectados y se pasó cuidadosamente el soldador sobre las patas del conector. Además, se utilizó malla de cobre para retirar el exceso de estaño presente entre algunos terminales.

Luego de realizar esta corrección, visualmente ya no se observaban cortocircuitos; sin embargo, al verificar nuevamente con un multímetro se detectó que el problema persistía en el conector USB de programación.

Con ayuda del profesor, se utilizó cinta térmica y una estación de aire caliente para calentar cuidadosamente el área del conector USB-C y reposicionarlo ligeramente hacia atrás. Esto permitió corregir la alineación del componente sobre la huella de la PCB, eliminando finalmente el cortocircuito detectado.

2. Huella incorrecta en uno de los switches

Se identificó que la huella seleccionada para uno de los switches no coincidía correctamente con el componente físico adquirido. Debido a esto, no fue posible montarlo de manera directa sobre la PCB.

La solución implementada fue realizar una conexión mediante cables, soldando manualmente los terminales necesarios para mantener la funcionalidad del interruptor dentro del circuito.

Luego de estas correcciones, la placa pudo ser energizada y utilizada para realizar pruebas básicas de funcionamiento.



Figura 7: Inspección y corrección de cortocircuitos en los conectores USB-C utilizando microscopio y soldador.

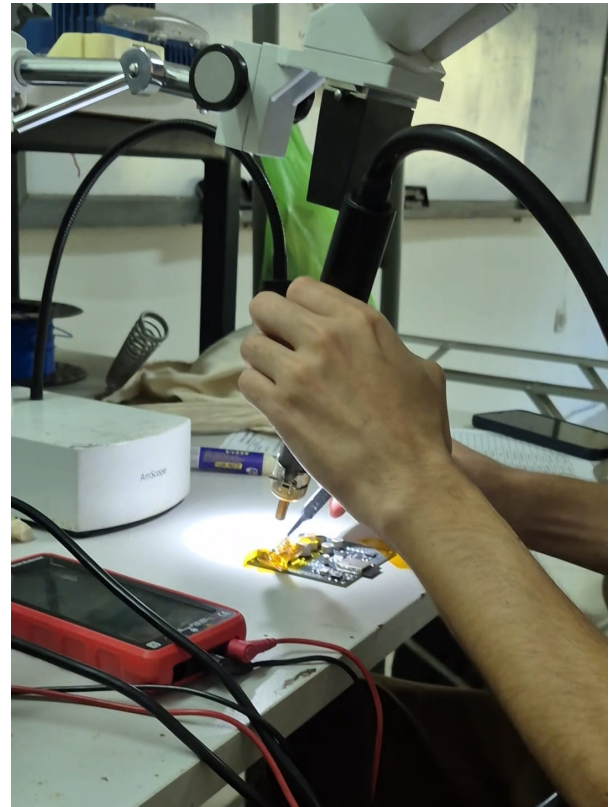


Figura 8: Corrección de cortocircuitos y reposicionamiento del conector USB-C de programación utilizando estación de aire caliente y cinta térmica.

9.3. Avances de programación (ESP-IDF)

9.3.1. Prueba 1: Parpadeo del LED de la placa (Blink del Pin 38)

```
1 // Incluimos las librerías principales
2 #include <stdio.h>
3 #include "freertos/FreeRTOS.h"
4 #include "freertos/task.h"
5 #include "driver/gpio.h"
6
7 #define BLINK_GPIO 38 // LED de la placa -> GPIO 38
8
9 static uint8_t estado = 0; // Estado actual del led
10
11 void app_main(void)
12 {
13     // Resetear y configurar el GPIO 38 como salida
14     gpio_reset_pin(BLINK_GPIO);
15     gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);
16
17     while (1) {
18         gpio_set_level(BLINK_GPIO, estado);
19         estado = !estado; // Cambia entre 0 y 1
20         // Delay de 500ms usando los ticks del RTOS
21         vTaskDelay(500 / portTICK_PERIOD_MS);
22     }
23 }
```

Listing 1: Código de parpadeo (Blink) en ESP-IDF

Utilizamos las siguientes librerías:

- **stdio.h:** librería estándar de entrada y salida de C.
- **freertos/FreeRTOS.h:** el núcleo del sistema operativo necesario para que el ESP32 pueda gestionar tareas y tiempo.
- **freertos/task.h:** librería específica de FreeRTOS para utilizar `vTaskDelay` (permite aplicar delays que no bloquean al procesador).
- **driver/gpio.h:** controlador de los pines GPIO; se utiliza para: resetear el estado actual del pin (`gpio_reset_pin`), configurar como entrada o salida (`gpio_set_direction`) y para definir el nivel lógico (HIGH o LOW) de la salida (`gpio_set_level`).

Con la variable estática *estado* se controla el nivel lógico del pin del led de la placa que es el GPIO 38.

9.3.2. Prueba 2: Pantalla LCD mediante comunicación I2C

```
1 #include <stdio.h>
2 #include "freertos/FreeRTOS.h"
3 #include "freertos/task.h"
4 #include "driver/i2c.h"
5 #include "driver/gpio.h"
6
7 // Definicion de pines I2C para ScramblerBot
8 #define SDA_PIN 1
9 #define SCL_PIN 2
10 #define I2C_PORT I2C_NUM_0
11 #define LCD_ADDR 0x20
12
13 // Funcion para enviar medio byte (nibble) manejando el pin ENABLE
14 void lcd_send_nibble(uint8_t data, uint8_t rs) {
15     uint8_t d = (data & 0xF0) | 0x08 | rs; // 0x08 enciende el backlight
16     uint8_t d_en = d | 0x04; // Pulso ENABLE HIGH
17     // Enviamos datos en I2C con EN = HIGH
18     i2c_master_write_to_device(I2C_PORT, LCD_ADDR, &d_en, 1, 100);
19     vTaskDelay(1 / portTICK_PERIOD_MS);
20     uint8_t d_dis = d & ~0x04; // Pulso ENABLE LOW
21     // Enviamos datos en I2C con EN = LOW
22     i2c_master_write_to_device(I2C_PORT, LCD_ADDR, &d_dis, 1, 100);
23     vTaskDelay(1 / portTICK_PERIOD_MS);
24 }
25
26 // Enviar byte completo separado en dos nibbles
27 void lcd_send_byte(uint8_t data, uint8_t rs) {
28     lcd_send_nibble(data, rs);
29     lcd_send_nibble(data << 4, rs);
30 }
31
32 // Comandos (rs = 0) y texto (rs = 1)
33 void lcd_cmd(uint8_t cmd) {
34     lcd_send_byte(cmd, 0);
35 }
36
37 void lcd_data(uint8_t data) {
38     lcd_send_byte(data, 1);
39 }
40
41 // Comando de set cursor del lcd
42 void lcd_set_cursor(uint8_t col, uint8_t row) {
43     // Direcciones de las filas
44     uint8_t row_offsets[] = {0x00, 0x40};
45     lcd_cmd(0x80 | (col + row_offsets[row])); // 0x80 -> mover cursor
46 }
47
48 // Imprimir texto
49 void lcd_print(const char *str) {
50     while (*str) {
51         lcd_data((uint8_t)(*str++));
52     }
53 }
```

```
52 }
53 // Inicializamos la pantalla
54 // Se envia tres veces la orden de despertar (0x30)
55 // 0x20 -> modo 4 bits
56 // 0x28 -> 4 bits y 2 lineas
57 void lcd_init() {
58     vTaskDelay(50 / portTICK_PERIOD_MS);
59     lcd_send_nibble(0x30, 0);
60     vTaskDelay(5 / portTICK_PERIOD_MS);
61     lcd_send_nibble(0x30, 0);
62     vTaskDelay(1 / portTICK_PERIOD_MS);
63     lcd_send_nibble(0x30, 0);
64     vTaskDelay(1 / portTICK_PERIOD_MS);
65     lcd_send_nibble(0x20, 0);
66     vTaskDelay(1 / portTICK_PERIOD_MS);
67     lcd_cmd(0x28);
68     lcd_cmd(0x0C); // Display ON
69     lcd_cmd(0x01); // Limpiar pantalla
70     vTaskDelay(2 / portTICK_PERIOD_MS);
71 }
72
73 void app_main(void)
74 {
75     // Configuración de periférico I2C
76     i2c_config_t conf = {
77         .mode = I2C_MODE_MASTER,
78         .sda_io_num = SDA_PIN,
79         .scl_io_num = SCL_PIN,
80         .sda_pullup_en = GPIO_PULLUP_ENABLE,
81         .scl_pullup_en = GPIO_PULLUP_ENABLE,
82         .master.clk_speed = 100000,
83     };
84     i2c_param_config(I2C_PORT, &conf);
85     i2c_driver_install(I2C_PORT, conf.mode, 0, 0, 0);
86
87     // Inicialización y escritura
88     lcd_init();
89     lcd_set_cursor(0, 0);
90     lcd_print("Hola soy ");
91     lcd_set_cursor(0, 1);
92     lcd_print("ScramblerBot!!");
93 }
```

Listing 2: Pantalla LCD 16x2 vía I2C en ESP-IDF

Además de las librerías ya utilizadas en la anterior prueba, usaremos la librería del controlador I2C (*driver/i2c.h*).

La idea principal de este programa es que se encarga de configurar el bus I2C para que el ESP32 controle una pantalla LCD mediante el envío directo de comandos binarios al expansor PCF8574. Inicializa el hardware manualmente en modo de 4 bits, activa la luz de fondo y gestiona la posición del cursor. Finalmente, traduce cadenas de texto en señales eléctricas para mostrar mensajes personalizados en ambas filas del display.

9.3.3. Prueba 3: Servomotores

```
1 #include <stdio.h>
2 #include "freertos/FreeRTOS.h"
3 #include "freertos/task.h"
4 #include "driver/ledc.h"
5
6 // Definicion de hardware y PWM
7 #define SERVO_PIN 7
8 #define LEDC_TIMER LEDC_TIMER_0
9 #define LEDC_MODE LEDC_LOW_SPEED_MODE
10 #define LEDC_CHANNEL LEDC_CHANNEL_0
11 #define LEDC_DUTY_RES LEDC_TIMER_14_BIT
12 #define LEDC_FREQUENCY 50
13
14 // Rangos de calibracion del servo MG996R
15 #define SERVO_MIN_PULSEWIDTH_US 500
16 #define SERVO_MAX_PULSEWIDTH_US 2400
17 #define SERVO_MAX_DEGREE 180
18
19 // Conversion de angulos a resolucion de Duty Cycle de 14 bits
20 uint32_t angle_to_duty(int angle) {
21     uint32_t pulsewidth = SERVO_MIN_PULSEWIDTH_US +
22         ((SERVO_MAX_PULSEWIDTH_US - SERVO_MIN_PULSEWIDTH_US) * angle) /
23         SERVO_MAX_DEGREE;
24
25     // (pulsewidth * resolucion_14bits) / periodo_20ms
26     return (pulsewidth * 16384) / 20000;
27 }
28
29 void app_main(void) {
30     // Configuracion de temporizador de 50Hz
31     ledc_timer_config_t ledc_timer = {
32         .speed_mode = LEDC_MODE,
33         .timer_num = LEDC_TIMER,
34         .duty_resolution = LEDC_DUTY_RES,
35         .freq_hz = LEDC_FREQUENCY,
36         .clk_cfg = LEDC_AUTO_CLK
37     };
38     ledc_timer_config(&ledc_timer);
39
40     // Configuracion de canal al pin GPIO 7
41     ledc_channel_config_t ledc_channel = {
42         .speed_mode = LEDC_MODE,
43         .channel = LEDC_CHANNEL,
44         .timer_sel = LEDC_TIMER,
45         .intr_type = LEDC_INTR_DISABLE,
46         .gpio_num = SERVO_PIN,
47         .duty = angle_to_duty(0),
48         .hpoint = 0
49     };
50     ledc_channel_config(&ledc_channel);
```

```
51 // Loop:
52 while (1) {
53     for (int pos = 0; pos <= SERVO_MAX_DEGREE; pos++) {
54         ledc_set_duty(LEDc_MODE, LEDC_CHANNEL, angle_to_duty(pos));
55         ledc_update_duty(LEDc_MODE, LEDC_CHANNEL);
56         vTaskDelay(15 / portTICK_PERIOD_MS);
57     }
58
59     for (int pos = SERVO_MAX_DEGREE; pos >= 0; pos--) {
60         ledc_set_duty(LEDc_MODE, LEDC_CHANNEL, angle_to_duty(pos));
61         ledc_update_duty(LEDc_MODE, LEDC_CHANNEL);
62         vTaskDelay(15 / portTICK_PERIOD_MS);
63     }
64 }
65 }
```

Listing 3: Control de Servomotor MG996R en ESP-IDF

La librería que se utiliza para el control del servomotor es una librería para controlar la salida PWM (*driver/ledc.h*).

Este programa configura un temporizador a 50Hz con resolución de 14 bits y traduce matemáticamente los ángulos (0° a 180°) en los anchos de pulso exactos (500µs a 2400µs) que exige el motor. Finalmente, ejecuta un bucle infinito que actualiza continuamente el ciclo de trabajo para realizar un barrido suave de ida y vuelta.

9.3.4. Simulación del algoritmo final

1. servo.h

```
1  #ifndef SERVO_H
2  #define SERVO_H
3  #include <stdint.h>
4  // Macros publicas del Servomotor
5  #define SERVO_1_PIN      7
6  #define SERVO_2_PIN      8
7  #define POS_INICIAL      0
8  #define POS_MEDIA        90
9  #define TIEMPO_MOV_MS    500 // Tiempo para que el servo llegue a
    su posicion
10
11 // Variable que puede usarse en el main.cpp
12 extern int contador_movimientos;
13
14 // Funciones publicas
15 void servo_init(void);
16 void mover_servos_secuencia(void);
17
18 #endif
19
```

Listing 4: servo.h

2. servo.cpp

```
1  #include "servo.h"
2  #include "driver/ledc.h"
3  #include "freertos/FreeRTOS.h"
4  #include "freertos/task.h"
5
6  // Macros internas de configuracion
7  #define LEDC_TIMER        LEDC_TIMER_0
8  #define LEDC_MODE         LEDC_LOW_SPEED_MODE
9  #define LEDC_CANAL_S1     LEDC_CHANNEL_0
10 #define LEDC_CANAL_S2     LEDC_CHANNEL_1
11 #define LEDC_RESOLUCION   LEDC_TIMER_14_BIT
12 #define LEDC_FRECUENCIA   50
13 #define PULSO_MIN         500
14 #define PULSO_MAX         2400
15 #define GRADOS_MAX        180
16
17 // Definicion de la variable extern
18 int contador_movimientos = 0;
19
20 // Convierte los angulos en Duty Cycles de 14 bits de resolucion
21 static uint32_t angle_to_duty(int angle) {
22     uint32_t pulsewidth = PULSO_MIN + ((PULSO_MAX - PULSO_MIN) *
angle) / GRADOS_MAX;
23     return (pulsewidth * 16384) / 20000;
24 }
```



```
25 // Movemos el servo y esperamos
26 static void mover_servo(ledc_channel_t canal, int grados) {
27     ledc_set_duty(LED_C_MODE, canal, angle_to_duty(grados));
28     ledc_update_duty(LED_C_MODE, canal);
29     vTaskDelay(TIEMPO_MOV_MS / portTICK_PERIOD_MS);
30 }
31 // Inicializamos el servo
32 void servo_init(void) {
33     ledc_timer_config_t timer_conf = {
34         .speed_mode = LED_C_MODE,
35         .duty_resolution = LED_C_RESOLUTION,
36         .timer_num = LED_C_TIMER,
37         .freq_hz = LED_C_FRECUENCIA,
38         .clk_cfg = LED_C_AUTO_CLK
39     };
40     ledc_timer_config(&timer_conf);
41
42     ledc_channel_config_t ch1_conf = {
43         .gpio_num = SERVO_1_PIN,
44         .speed_mode = LED_C_MODE,
45         .channel = LED_C_CANAL_S1,
46         .intr_type = LED_C_INTR_DISABLE,
47         .timer_sel = LED_C_TIMER,
48         .duty = angle_to_duty(POS_INICIAL),
49         .hpoint = 0
50     };
51     ledc_channel_config(&ch1_conf);
52
53     ledc_channel_config_t ch2_conf = {
54         .gpio_num = SERVO_2_PIN,
55         .speed_mode = LED_C_MODE,
56         .channel = LED_C_CANAL_S2,
57         .intr_type = LED_C_INTR_DISABLE,
58         .timer_sel = LED_C_TIMER,
59         .duty = angle_to_duty(POS_INICIAL),
60         .hpoint = 0
61     };
62     ledc_channel_config(&ch2_conf);
63 }
64 void mover_servos_secuencia(void) {
65     // Se mueve el servo 1 a 90 grados
66     mover_servo(LED_C_CANAL_S1, POS_MEDIA);
67     // Se mueve el servo 2 a 90 grados
68     mover_servo(LED_C_CANAL_S2, POS_MEDIA);
69     // Vuelve a posicion original el servo 2
70     mover_servo(LED_C_CANAL_S2, POS_INICIAL);
71     // Vuelve a posicion original el servo 1
72     mover_servo(LED_C_CANAL_S1, POS_INICIAL);
73     contador_movimientos++;
74 }
75
```

Listing 5: servo.cpp

3. lcd.h

```
1  #ifndef LCD_H
2  #define LCD_H
3  #include <stdint.h>
4  // Macros publicas de la pantalla lcd
5  #define SDA_PIN      1
6  #define SCL_PIN      2
7  #define I2C_PORT      I2C_NUM_0
8  #define LCD_ADDR      0x20
9
10 // Funciones publicas a utilizar en el main.cpp
11 void lcd_init(void);
12 void lcd_set_cursor(uint8_t col, uint8_t row);
13 void lcd_print(const char *str);
14 void lcd_clear(void);
15
16 #endif
17
```

Listing 6: lcd.h

4. lcd.cpp

```
1  #include "lcd.h"
2  #include "driver/i2c.h"
3  #include "freertos/FreeRTOS.h"
4  #include "freertos/task.h"
5
6  // Enviar 4 bits a la pantalla lcd
7  static void lcd_send_nibble(uint8_t data, uint8_t rs) {
8      uint8_t d = (data & 0xF0) | 0x08 | rs;
9      uint8_t d_en = d | 0x04;
10     i2c_master_write_to_device(I2C_PORT, LCD_ADDR, &d_en, 1, 100)
11     ;
12     vTaskDelay(1 / portTICK_PERIOD_MS);
13     uint8_t d_dis = d & ~0x04;
14     i2c_master_write_to_device(I2C_PORT, LCD_ADDR, &d_dis, 1,
15     100);
16     vTaskDelay(1 / portTICK_PERIOD_MS);
17 }
18
19 // Enviar byte entero a la pantalla lcd
20 static void lcd_send_byte(uint8_t data, uint8_t rs) {
21     lcd_send_nibble(data, rs);
22     lcd_send_nibble(data << 4, rs);
23 }
24
25 // Comando (rs=0) y texto (rs=1)
26 static void lcd_cmd(uint8_t cmd) {
27     lcd_send_byte(cmd, 0);
28 }
29
30 static void lcd_data(uint8_t data) {
31     lcd_send_byte(data, 1);
32 }
33
```

```
30
31 void lcd_set_cursor(uint8_t col, uint8_t row) {
32     uint8_t row_offsets[] = {0x00, 0x40};
33     lcd_cmd(0x80 | (col + row_offsets[row]));
34 }
35
36 void lcd_print(const char *str) {
37     while (*str) {
38         lcd_data((uint8_t)(*str));
39         str++;
40     }
41 }
42
43 void lcd_clear(void) {
44     lcd_cmd(0x01);
45     vTaskDelay(2 / portTICK_PERIOD_MS);
46 }
47
48 // Funcion de inicializacion
49 void lcd_init() {
50     i2c_config_t conf = {
51         .mode = I2C_MODE_MASTER, .sda_io_num = SDA_PIN, .
scl_io_num = SCL_PIN,
52         .sda_pullup_en = GPIO_PULLUP_ENABLE, .scl_pullup_en =
GPIO_PULLUP_ENABLE,
53         .master = {.clk_speed = 100000}
54     };
55     i2c_param_config(I2C_PORT, &conf);
56     i2c_driver_install(I2C_PORT, conf.mode, 0, 0, 0);
57
58     vTaskDelay(50 / portTICK_PERIOD_MS);
59     lcd_send_nibble(0x30, 0);
60     vTaskDelay(5 / portTICK_PERIOD_MS);
61     lcd_send_nibble(0x30, 0);
62     vTaskDelay(1 / portTICK_PERIOD_MS);
63     lcd_send_nibble(0x30, 0);
64     vTaskDelay(1 / portTICK_PERIOD_MS);
65     lcd_send_nibble(0x20, 0);
66     vTaskDelay(1 / portTICK_PERIOD_MS);
67
68     lcd_cmd(0x28);
69     lcd_cmd(0x0C); // Display ON
70     lcd_clear();
71 }
72
```

Listing 7: lcd.cpp

5. main.cpp

```
1  #include <stdio.h>
2  #include "freertos/FreeRTOS.h"
3  #include "freertos/task.h"
4  #include "lcd.h"
5  #include "servo.h"
6  // Macros de tiempos
7  #define TIEMPO_SALUDO_MS 5000 // 5 segundos
8  #define TIEMPO_RESOLVIENDO_MS 10000 // 10 segundos
9
10 // El 'extern "C"' para que reconozca la funcion principal
11 extern "C" void app_main(void)
12 {
13     // Inicializacion del hardware
14     lcd_init();
15     servo_init();
16     // Mensaje inicial
17     lcd_clear();
18     lcd_set_cursor(0, 0);
19     lcd_print("Hola soy ");
20     lcd_set_cursor(0, 1);
21     lcd_print("ScramblerBot!!");
22     vTaskDelay(TIEMPO_SALUDO_MS / portTICK_PERIOD_MS);
23     // Tomamos la marca de tiempo actual en milisegundos
24     TickType_t tiempo_inicio = xTaskGetTickCount() *
portTICK_PERIOD_MS;
25     // Mientras no hayan pasado 10 segundos, repetimos la
secuencia
26     while (((xTaskGetTickCount() * portTICK_PERIOD_MS) -
tiempo_inicio) < TIEMPO_RESOLVIENDO_MS) {
27         \\ Imprimimos en la pantalla
28         lcd_clear();
29         lcd_set_cursor(0, 0);
30         lcd_print("Resolviendo...");
31         lcd_set_cursor(0, 1);
32         lcd_print("Movimientos: ");
33         lcd_set_cursor(13, 1);
34         lcd_print(contador_movimientos);
35
36         mover_servos_secuencia();
37     }
38     // Pausa
39     vTaskDelay(TIEMPO_PAUSA_MS / portTICK_PERIOD_MS);
40     // Mensaje final
41     lcd_clear();
42     lcd_set_cursor(0, 0);
43     lcd_print("Cubo Resuelto!!");
44 }
45
```

Listing 8: main.cpp

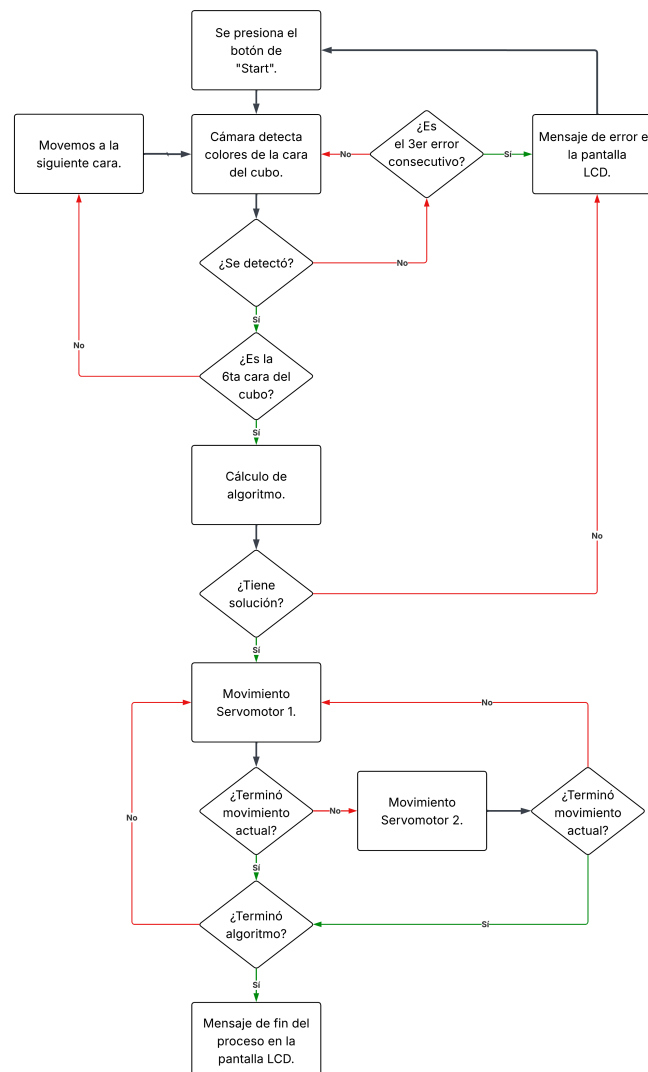
9.4. Flujo conceptual del algoritmo

El flujo conceptual del algoritmo final es presentado mediante el siguiente diagrama de flujo. El sistema inicia cuando el usuario presiona el botón de inicio, momento en el cual la cámara comienza la detección de colores de cada cara del cubo Rubik.

Si una cara no es detectada correctamente, el sistema realiza nuevos intentos y, en caso de alcanzar múltiples errores consecutivos, se muestra un mensaje de error en la pantalla LCD.

Una vez detectadas las seis caras del cubo, el ESP32-S3 procesa la información obtenida y calcula la secuencia de movimientos necesaria para resolver el cubo. Posteriormente, se ejecutan los movimientos de los servomotores de manera secuencial hasta completar el algoritmo.

Finalmente, el sistema muestra en la pantalla LCD un mensaje indicando la finalización del proceso.



9.5. Resultados de pruebas aisladas

Se realizaron pruebas aisladas para verificar el funcionamiento de algunos módulos principales de la PCB.

1. Prueba de pantalla LCD

Se logró programar la pantalla LCD y mostrar el mensaje:

“Hola soy ScramblerBot!!”

Esta prueba permitió validar la alimentación de la pantalla y la comunicación mediante el bus I2C.

2. Prueba de LED indicador

Se programó correctamente un LED de prueba conectado al pin 38 del ESP32-S3. Esto permitió verificar la configuración de GPIO como salida digital y comprobar que el microcontrolador ejecuta el firmware cargado.

3. Prueba de servomotor

Se conectó un servomotor MG996R a una de las salidas PWM de la placa y se logró generar movimiento mediante programación. Esta prueba permitió validar la etapa de control PWM y la conexión hacia los actuadores.



Figura 9: Prueba de funcionamiento de la pantalla LCD mediante comunicación I2C.

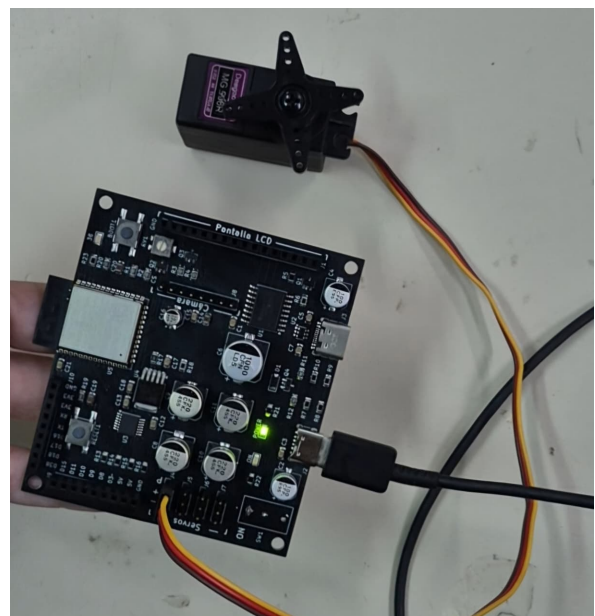


Figura 10: Prueba de control PWM utilizando un servomotor MG996R.

Los resultados parciales indican que la placa responde correctamente en pruebas básicas de salida digital, comunicación con pantalla LCD y control de actuadores mediante PWM.